

PYTHON

Module 15: Immutable Tuples & Unpacking

CODING

Introduction to Tuples

Tuples are a fundamental data structure in Python, used to store multiple items in a single variable. They are characterized by their round brackets **()** and their **immutable nature**.

Core Properties at CodeHive:

- ****Ordered****: The items have a defined order that will not change.
- ****Unchangeable (Immutable)****: Once created, you cannot add, remove, or modify items.
- ****Allows Duplicates****: Since items are indexed, you can have multiple items with the same value.

The Tuple Singularity

A common pitfall for beginners is creating a tuple with a single item. To recognize it as a tuple, Python requires a trailing comma.

• Proper Syntax: `single_item = ("apple",)`

• Incorrect Syntax: `not_a_tuple = ("apple")` (This is treated as a standard string).

Functional Note: Tuples can store any data type, and a single tuple can even contain a mix of strings, integers, and booleans.

Accessing and Slicing

You access tuple items using the same index-based logic as lists.

- Positive Indexing: `thistuple[1]` accesses the second item.

- Negative Indexing: `thistuple[-1]` accesses the last item.

- Range Slicing: `thistuple[2:5]` extracts a subsection. Note that the start is inclusive and the end is exclusive.

- Membership: Use the `in` keyword to check if a value exists within the collection.

Immutable Workarounds

Even though tuples are technically unchangeable, developers often use a 'Conversion Workaround' to update data.

Process:

1. Convert the tuple to a list: `temp_list = list(mytuple)`

2. Modify the list items normally.

3. Convert the list back to a tuple: `mytuple = tuple(temp_list)`

This strategy allows you to bypass immutability while maintaining the structural benefits of a tuple for the rest of your program.

Packing and Unpacking

Python offers 'Unpacking', allowing you to extract tuple values directly into separate variables.

- Standard Unpacking: `(green, yellow, red) = fruits`

- Using Asterisks (*): If the number of variables is less than the items, use an asterisk to capture the remaining items as a list.

Example: `(first, *middle, last) = long_tuple` will put everything between the first and last elements into a list named `middle`.

Iteration and Joining

• **Looping:** You can navigate through a tuple using `for` loops or `while` loops (using `len()` and index counters).

• **Joining:** Use the `+` operator to concatenate two tuples into a new third tuple.

• **Multiplying:** Use the `*` operator to repeat the contents of a tuple multiple times within a new collection.

Tuple Methods Reference

Because tuples are immutable, they only feature two built-in methods:

- `count()` : Returns the frequency of a specified value.

- `index()` : Returns the first index position where a specified value is found.

CodeHive Pro-Tip: Use tuples for data that should not change throughout your execution, such as geographic coordinates or database configuration settings.